

Computing IC for REx Pipeline

1 Overview

This document describes the process for generating the Information Content (IC) file used in the REx pipeline. The file was created outside the REx pipeline itself so REx will work with any JSON file that follows the required format.

Important: The complete process for computing node/path relevance is documented in our paper. This document provides a technical overview and implementation details.

2 Step-by-Step Process

2.1 Graph Expansion

Extended the base dataset using ontology mappings via AML (AgreementMakerLight). Take Hetionet as an example:

- Hetionet is a "flat ontology" lacking rich semantic information.
- Connected Hetionet to CHEBI and NCIT ontologies for additional context (ontologies used for the calculation of the lowest common ancestors).
- This enrichment enables the generation of more informative embeddings.

2.2 Embedding Generation

Generated embeddings using OWL2Vec* (OWL2Vec-Star) on the ontology-enriched graph.

Note: Alternative embedding methods can be used depending on your dataset. If ontology enrichment is skipped, consider methods more suitable for your graph.

2.3 Clustering

Applied K-means clustering using FAISS (FAISS)

- Number of clusters = 10% of the total node count

2.4 Cluster-Aware Node Degree Calculation

Computed modified node degrees accounting for cluster membership:

- Entities in the same cluster are counted as one connection
- **Example:** If Drug A has a degree of 3, but two connected entities share a cluster, the adjusted degree becomes 2.
- Calculations performed separately for each edge type (e.g., only counting “related_to” relations)

See Figure 1 from the IJCAI2025 presentation for a better understanding.

Correcting Research Bias with Clustered IC

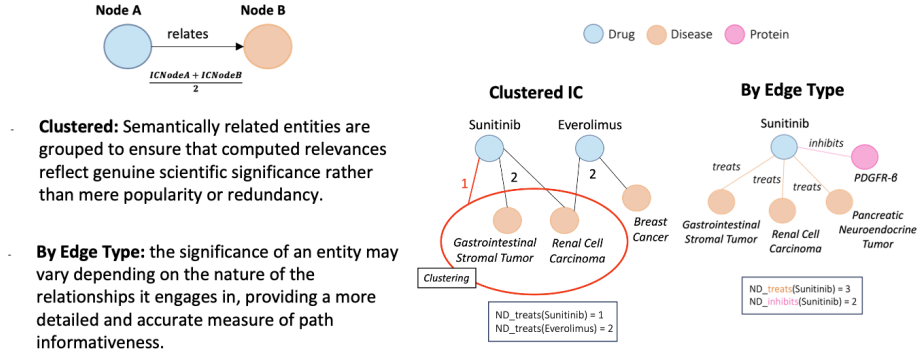


Figure 1: Computing IC example from IJCAI2025 presentation.

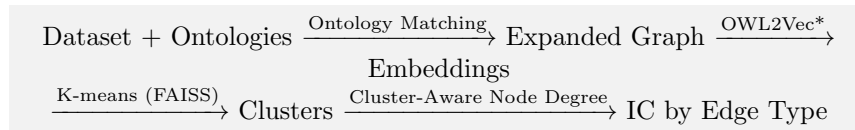
2.5 IC Computation

Applied the IC_{node} measure to the cluster-adjusted node degrees, categorized by edge type.

The normalization formula used is:

$$IC_{normalized} = \frac{-\log_2(degree/max_degree)}{-\log_2(1/max_degree)}$$

3 Process Pipeline



4 Implementation Resources

Example scripts are provided in the `Computing_IC` folder. These three Python scripts work in sequence:

1. **get_embs_byEdgeType.py**: Organizes embeddings by edge type
 - Reads embeddings from JSON file
 - Extracts edges from RDF graph using RDFLib
 - Groups embeddings by predicate/edge type
 - Outputs dictionary with edge URIs as keys
2. **faiss_embs_edgeType.py**: K-means clustering implementation
 - Uses FAISS for efficient large-scale clustering
 - Automatically sets centroids to 10% of node count
 - Processes each edge type independently
 - Optional histogram visualization of cluster distribution
3. **calculate_clusterIC.py**: Computes Information Content
 - Calculates cluster-aware node degrees
 - Applies normalization formula for IC computation
 - Filters out annotation and datatype properties (in case of complex ontologies)
 - Outputs IC values by edge type

4.1 Input/Output Formats

4.1.1 Input: Embeddings Format

The expected embedding format is a JSON file with entity URIs as keys and embedding vectors as values:

```
{
  "http://purl.obolibrary.org/obo/NCIT_C181743":
    ["0.16179", "-0.03999", "0.14161", ...],
  "http://www.phoc.org.cn/pmo/class/PMO_00074795":
    ["0.26419", "-0.10048", "0.11824", ...],
  "https://genenames.org/hgnc_id/HGNC:12333":
    ["0.06358", "-0.25567", "0.31913", ...]
}
```

4.1.2 Output from Step 1: Embeddings by Edge Type

After organizing by edge type, embeddings are grouped by predicate:

```
{
  "http://www.w3.org/2000/01/rdf-schema#subClassOf": {
    "http://purl.obolibrary.org/obo/NCIT_C181743":
      ["0.16179", "-0.03999", ...],
    "http://purl.obolibrary.org/obo/G0_0005898":
      ["0.85866", "0.03567", ...],
    "http://purl.obolibrary.org/obo/CHEBI_17186":
      ["0.14161", "-0.11844", ...],
    ...
  },
  "http://purl.obolibrary.org/obo/R0_0002170": {
    "http://purl.obolibrary.org/obo/G0_0008558":
      ["0.26345", "-0.13813", ...],
    ...
  }
}
```

4.1.3 Output from Step 2: Clustering Results

After clustering, entities are grouped by cluster ID within each edge type:

```
{
  "http://www.w3.org/2000/01/rdf-schema#subClassOf": {
    "0": [
      "http://purl.obolibrary.org/obo/G0_0005898",
      "http://purl.obolibrary.org/obo/G0_1990620",
      "http://www.phoc.org.cn/pmo/class/PM0_00052348"
    ],
    "1": [
      "http://purl.obolibrary.org/obo/G0_0008558",
      "http://purl.obolibrary.org/obo/G0_0033266"
    ],
    "2": [
      "http://purl.bioontology.org/ontology/LNC/LP33037-0",
      "http://purl.bioontology.org/ontology/LNC/LP33042-0"
    ]
  },
  "http://another/predicate/uri": {
    "0": [...],
    "1": [...]
  }
}
```

Format Requirements:

- Each entity URI maps to its embedding vector (as string-formatted floats)
- All embeddings must have the same dimensionality (typically 100-300 dimensions)
- Entity URIs should match those in your RDF graph
- Cluster IDs are strings representing integers

4.2 Requirements

4.2.1 Input Files

- **embeddings.json**: Node embeddings in the format shown above
- **graph.owl**: Input graph in RDF format (RDFLib library used)
- **edge_labels.tsv**: TSV file with edge URIs and labels
- **owlClasses.txt**: List of OWL class URIs, meaning the entity nodes (one per line)

4.3 Notes

- These scripts were developed for research purposes and may require adjustment for your specific case
- Scripts include checkpoint logging for debugging large datasets
- All scripts accept alternative embedding methods as long as the JSON format is maintained
- Intermediate files may be saved at each step for inspection